

SYSTEM ARCHITECTURE DOCUMENT

Katuwang

A Web-Based Peer Tutoring & Academic Mentoring Platform
Taysan Resettlement Integrated School (TRIS) — Grades 7–12

Document	Finalized Architecture Design
Version	1.0 (baseline)
Date	8 September 2026
Status	Finalized — reflects the implemented system on <code>main</code>
Scope	Modules 1–6, all three roles, data & security architecture

Table of Contents

1. Introduction & Purpose

- Objectives
- Audience
- Reference Documents

2. System Overview

3. Architectural Drivers

- Quality Attributes
- Constraints
- Assumptions

4. Architectural Style & Guiding Principles

5. Technology Stack

6. Logical Architecture — Layered View

- Presentation Layer
- API / Route-Handler Layer
- Domain & Service Layer
- Data-Access Layer
- Persistence Layer

7. Module / Component View

- Module-to-Code Map
- Cross-Module Dependencies

8. Runtime Views (Key Scenarios)

- Registration
- Authentication & RBAC Enforcement
- Tutor Matching
- Tutor Certification Assessment
- Session Pre-/Post-Test

9. Data Architecture

- Entity Groups
- Normalization & Denormalization
- Anonymous Identity Generation
- Entity-Relationship Diagram

10. Security & Privacy Architecture

- Authentication
- Authorization
- Double-Blind Anonymity (RA 10173)
- Account Lifecycle
- Auditability

11. Cross-Cutting Concerns

12. Deployment & Infrastructure View

13. Source Tree Layout

14. Architecture Decision Record (Summary)

15. Non-Functional Considerations & Known Limitations

16. Appendix A — Data Model Inventory

17. Appendix B — API Surface Summary

1. Introduction & Purpose

This document is the finalized architecture baseline for **Katuwang** (Tagalog for “*helper*” / “*partner*”), a free, non-profit, web-based peer-tutoring management platform built for Taysan Resettlement Integrated School (TRIS), a Philippine public secondary school operating under classroom overcrowding and split shifts. It describes the system as *actually built* on the `main` branch, not the original thesis proposal.

1.1 Objectives

- Give a single authoritative description of the runtime, logical, data, and security architecture.
- Record the architectural style and the decisions that shaped it, with rationale.
- Serve as the onboarding reference for developers and the evaluation artifact for the capstone defense.

1.2 Audience

Project developers and maintainers, the capstone advisory panel, and TRIS technical stakeholders responsible for future operation.

1.3 Reference Documents

Document	Role
CLAUDE.md	Engineering conventions, stack table, RBAC/API patterns
docs/reference/project-overview.md	Domain / business context digest
docs/reference/decisions.md	Authoritative record of divergences from the thesis proposal
docs/feature-checklist.md	Thesis-spec vs. implemented status per module
docs/reference/auth-implementation.md	Auth build guide (NextAuth, RBAC, anonymous IDs)
docs/erd.md / prisma/schema.prisma	Generated ERD and the schema source of truth
docs/roles/{LEARNER, TUTOR, ADMIN}.md	Per-role feature & API reference

2. System Overview

Katuwang connects **student tutors** (peers who volunteer to teach) with **student learners** (peers who need help) so that tutoring can be organized, scheduled, assessed, and moderated instead of being ad hoc. Everything is voluntary and free — there is no payment or monetization anywhere in the system, and no upload/storage of learning materials because tutoring physically happens on school premises.

The platform is delivered as a **single full-stack Next.js 16 application** (App Router) backed by a MariaDB/MySQL database. It exposes three role-scoped portals from one deployable unit:

Role (Prisma Role)	Portal	Core capability
STUDENT_LEARNER	/learner	Browse/enroll in classes, get matched to tutors, post class requests, take pre/post-tests, view progress
STUDENT_TUTOR	/tutor	Create/run classes & sessions, get certified per topic, fulfill class requests, build session tests
ADMIN	/admin	User & class moderation, question bank, certification review, subjects/topics, settings, analytics, audit log

The system is scoped into **six core modules**: (1) User Management, (2) Session Management, (3) Tutor Matching, (4) Assessment, (5) Chatbot Assistant, and (6) Analytics Dashboard. All six are implemented; the chatbot is a deterministic rule-based intent matcher (no LLM), consistent with the thesis delimitation.

Scope note. The thesis proposal named a fourth “Teacher Moderator” role. It was intentionally cut — the platform ships with exactly three roles and moderation duties fold into `ADMIN`. See `docs/reference/decisions.md`.

3. Architectural Drivers

3.1 Quality Attributes (ranked)

#	Attribute	Why it drives the architecture
1	Privacy / Confidentiality	RA 10173 mandates double-blind anonymity between peers. Drives the anonymous-ID scheme, peer-facing DTO discipline, and an admin-gated real-name override.
2	Security	Minors' data. Drives edge RBAC + per-route re-checks, bcrypt hashing, JWT sessions, account lifecycle gating at login, and an immutable audit log.
3	Maintainability	Student-built, must be handed over. Drives one language end-to-end (TypeScript), one ORM, lean Server Components, service logic in <code>src/lib/</code> , and Zod-validated boundaries.
4	Correctness / Integrity	Enrollment caps, certification gates, request lifecycles. Drives Prisma transactions, compound unique constraints, and pure testable domain functions.
5	Usability & Accessibility	Low-resource devices. Drives responsive DaisyUI/Tailwind layout, server rendering, and no heavy client frameworks.
6	Deployability	Minimal ops capacity at TRIS. Drives a single monolith, no background workers, no message broker, no external service hard-dependency.

3.2 Constraints

- **Regulatory:** RA 10173 (Data Privacy Act) — parental/guardian consent captured at registration; peer identities never crossed.
- **Non-profit:** no billing, payment, or monetization code anywhere.
- **No file storage:** no upload endpoints or blob storage — tutoring materials are handled in person.
- **No scheduler/queue infrastructure:** all work is request-driven; there is no cron, SSE, polling worker, or background job runner. Time-based notifications (e.g. “class starts in 1 h”) are therefore out of scope.
- **Chatbot:** nav/FAQ/recommendation only, deterministic, no generative model, no new runtime dependency.
- **Analytics:** descriptive aggregates only — no ML, prediction, or educational data mining.
- **Three roles only** (see §2).

3.3 Assumptions

- Single institution (TRIS); no multi-tenancy.
- School-day-length engagement — JWT sessions expire after 8 hours.
- SMTP is available for password-reset mail; when unset, the app logs the link to the console (dev fallback).
- Modest concurrent load (hundreds of students), well within a single Node process + one MariaDB instance.

4. Architectural Style & Guiding Principles

Katuwang is a **modular monolith** built on the **Next.js App Router**, combining React Server Components (RSC) for data-fetching pages with client “islands” for interactivity, plus co-located REST route handlers for mutations.

Principle	Application in Katuwang
Server-authoritative	Every privileged decision is re-checked on the server (<code>getSession</code> + role guard in each route handler). UI hiding is cosmetic only.
Lean pages, thick components	<code>src/app/**/page.tsx</code> are Server Components that fetch session + initial data; interaction/state lives in <code>"use client"</code> components under <code>src/components/</code> .
Domain logic is pure & centralized	Ranking, grading, access rules, serialization, config resolution live as framework-free functions in <code>src/lib/</code> and are unit-tested in isolation.
Validated boundaries	All request bodies parse through Zod schemas (<code>src/lib/validations/</code>); first validation error returns HTTP 400.
One ORM, one client	All DB access goes through the singleton <code>prisma</code> (<code>@/lib/prisma</code>); multi-table writes are wrapped in <code>prisma.\$transaction</code> .
Feature flags over branches	Runtime behavior toggled by <code>PlatformSetting</code> rows (e.g. <code>matchingEnabled</code> , <code>chatbotEnabled</code>) read through <code>src/lib/settings.ts</code> .
Privacy by construction	Peer-facing queries <code>select only id + anonymousId</code> (+ non-identifying fields); real names require an admin route or the explicit <code>showTutorRealNames</code> override.

5. Technology Stack

Layer	Technology	Version	Role
Framework	Next.js (App Router)	16.2	Full-stack: RSC pages, client components, REST route handlers, edge middleware
UI runtime	React	19.2	Component model, server + client rendering
Language	TypeScript	5.x	Strict static typing across models, validation, APIs
Styling	Tailwind CSS 4 + DaisyUI 5	4 / 5	OKLCH semantic theme tokens, responsive layout
Icons	lucide-react	1.x	Action / status / nav iconography
Charts	Recharts	3.x	Admin analytics & learner progress visualizations
Auth	NextAuth.js	4.24	Credentials provider, JWT session strategy
ORM	Prisma	7.8	Schema modeling, migrations, typed queries
Database	MariaDB / MySQL	MariaDB 3.5 driver	Relational persistence via <code>@prisma/adapters-mariadb</code>
Validation	Zod	4.x	Runtime request validation, discriminated unions
Hashing	bcrypt / bcryptjs	6 / 3	One-way password hashing (cost factor 12)
Mail	Nodemailer	7.x	SMTP password-reset mail, console fallback when unconfigured
Testing	Vitest	4.x	Unit + route-handler integration tests
Package manager	pnpm	—	Dependency management with pinned overrides

6. Logical Architecture — Layered View

1 — Presentation Layer

Server Components (`src/app/**/page.tsx`) fetch session + initial data · Client Components (`src/components/**` , "use client") hold interaction/state · `PortalLayout` shell, DaisyUI theme, Recharts, chatbot widget

↓ HTTP / RSC payload ↑

2 — Edge Middleware (`src/proxy.ts`)

Runs before every `/dashboard` , `/admin` , `/tutor` , `/learner` request · requires a valid JWT · enforces role-to-path prefix, redirects to `/unauthorized`

↓

3 — API / Route-Handler Layer (`src/app/api/**/route.ts`)

~55 REST endpoints · per-route `getSession` + role check · Zod body validation · standard `{ error }` / payload response contract · HTTP 400/401/403/404/409/500

↓

4 — Domain & Service Layer (`src/lib/**`)

Pure logic: `matching.ts` (weighted ranking), `assessmentPicker / gradeAttempt` (quiz build & grading), `sessionTestAccess / sessionTestResults`, `moderation.ts`, `notifications.ts`, `settings.ts` (feature flags), `subjects.ts` (taxonomy + cache), `idGenerator.ts`, `auditLog.ts`, `mail.ts`, `chatbot/*` (tokenise → intent scoring)

↓

5 — Data-Access Layer (`src/lib/prisma.ts`)

Singleton Prisma client · `$transaction` for multi-table writes · compound unique keys · generated types shared with the app

↓ SQL (MariaDB adapter)

6 — Persistence Layer — MariaDB / MySQL

28 tables · enum-backed status columns · 3NF core with deliberate denormalization of `topic` strings · migration history under `prisma/migrations/`

6.1 Presentation Layer

Pages are Server Components by default: they call `getSession(authOptions)` , load only the data the first paint needs, and pass it as props into client components. Client components own forms, modals, tables with client-side sort/filter, and optimistic UI. The shared `PortalLayout` provides the role-aware sidebar, top-bar global search (🔍), and the notification bell.

6.2 API / Route-Handler Layer

Each mutation or protected read is a route handler under `src/app/api/` . The canonical shape is: authenticate → authorize by role → Zod-parse → execute (often inside a transaction) → return payload or `{ error }` . Server errors are logged with `console.error` and returned as a generic 500 so internals never leak.

6.3 Domain & Service Layer

Business rules live here as framework-free modules so they can be reasoned about and unit-tested without HTTP or a database.

Examples: `rankMatches()` scores classes for a learner using explicit tunable weights (topic overlap, certified-topic bonus, schedule fit, grade proximity, soonness tiebreak); `gradeAttempt` grades an assessment; `sessionTestAccess` decides whether a learner may open a PRE or POST attempt given the session lifecycle.

6.4 Data-Access Layer

A single Prisma client instance is reused across hot reloads via a `globalThis` guard. All dependent multi-row writes (e.g. tutor registration creating `User` + `TutorProfile` ; accepting a class request creating a `TutorClass` and transitioning the request) are wrapped in `prisma.$transaction` for atomicity.

6.5 Persistence Layer

MariaDB via the Prisma MariaDB adapter. Status fields use Prisma enums for integrity; the subject taxonomy moved from a compile-time enum to admin-editable `Subject / Topic` tables, with `subject` columns storing the immutable slug as a plain string.

7. Module / Component View

7.1 Module-to-Code Map

Module	Primary models	Key API roots	Domain logic	Status
1. User Management	User, TutorProfile, PasswordResetToken, IdCounter	/api/register , /api/auth/* , /api/{learner,tutor}/profile , /api/admin/users , /api/admin/registrations	idGenerator , registration , passwordReset , moderation	Complete
2. Session Management	TutorClass, ClassTopic, ClassSession, ClassEnrollment, ClassAppeal	/api/tutor/classes/** , /api/classes/[id]/enroll , /api/admin/classes , /api/admin/class-appeals	classQueries , classSessions , classBrowserCache , derivedAvailability	Complete
3. Tutor Matching	TopicRequest, TopicRequestTopic, TopicRequestSlot, Notification	/api/learner/match , /api/learner/topic-requests , /api/tutor/topic-requests/[id]/accept , /api/admin/topic-requests	matching (rankMatches) , browseRanking , topicRequestVisibility , notifications	Complete
4. Assessment	AssessmentQuestion, AssessmentOption, AssessmentAttempt(+Item), TopicCertification, QuestionRequest, SessionTest, SessionTestQuestion, SessionTestAttempt(+Item)	/api/tutor/assessments/** , /api/tutor/.../sessions/[id]/test/** , /api/learner/session-test-attempts/** , /api/admin/assessment-*, /api/admin/session-tests/**	assessmentPicker , assessmentConfig , assessmentStatus , gradeAttempt , sessionTestAccess , sessionTestResults	Complete (tutor certification + per-session learner pre/post-test)
5. Chatbot Assistant	ChatbotMiss	/api/chatbot , /api/admin/chatbot-misses	chatbot/classifier , chatbot/intents , chatbot/faq , chatbot/recommend	Complete (deterministic, no LLM)
6. Analytics Dashboard	(read-only aggregates), AuditLog	/api/admin/reports , /api/admin/audit-log	in-route aggregation, auditLog	Complete (descriptive only)
<i>Cross-cutting: Taxonomy</i>	Subject, Topic	/api/subjects , /api/admin/subjects/** , /api/admin/topics/**	subjects (cache + fallback), subjectTopics (normalizeTopic)	Complete
<i>Cross-cutting: Settings & Notifications</i>	PlatformSetting, Notification	/api/admin/settings , /api/notifications	settings , notifications	Complete

7.2 Cross-Module Dependencies

- Matching** → **Session Management**: rankMatches reads live TutorClass + ClassSession + enrollment counts; accepting a TopicRequest creates a TutorClass.
- Matching** → **Assessment**: ranking gives a bonus for topics the teaching tutor holds a CERTIFIED TopicCertification for.
- Assessment (session tests)** → **Question Bank**: session-test questions are drawn from AssessmentQuestion filtered to origin: BANK plus the tutor's own origin: TUTOR questions; the tutor certification quiz pool is bank-only.
- Chatbot** → **Matching**: the learner "recommend" intent calls the same rankMatches engine.
- All modules** → **Notifications & Audit**: lifecycle events emit Notification rows; every admin moderation action writes an AuditLog row.
- All modules** → **Settings**: matchingEnabled, chatbotEnabled, sessionTestsEnabled, requireRegistrationApproval, requireCertificationForClassCreation, autoCertifyOnAssessmentPass, registrationOpen, showTutorRealNames gate behavior at runtime.

8. Runtime Views (Key Scenarios)

8.1 Registration (learner or tutor)

1. Client form (`LearnerRegisterForm` / `TutorRegisterForm`) does local validation, then `POST /api/register` with a `type` discriminator.
2. Route handler `Zod`-validates, enforces guardian `consentGiven`, checks e-mail uniqueness (409 on conflict).
3. `bcrypt.hash(password, 12)`; `generateAnonymousId("LEARNER"|"TUTOR")` atomically increments the `IdCounter` row (`STU-####` / `TUT-####`).
4. `prisma.$transaction` creates `User` (+ `TutorProfile` for tutors). If `requireRegistrationApproval` is on, status is `PENDING`; otherwise `ACTIVE`.
5. 201 with the new anonymous ID; client redirects to `/login` (or `/pending-approval`).

8.2 Authentication & RBAC Enforcement

1. `signIn("credentials", ...)` → `NextAuth authorize()` in `src/lib/auth.ts`.
2. Look up user by e-mail, `bcrypt.compare` the password.
3. Account-status gate: expired `SUSPENDED` auto-reactivates; active `SUSPENDED` / `BANNED` rejected; `PENDING` / `DECLINED` return sentinel errors that the login page maps to `/pending-approval` / `/account-declined`.
4. On success, `jwt` callback embeds `id`, `anonymousId`, `role`, `fullName`; session strategy is `JWT`, `maxAge` 8 h.
5. Every subsequent protected request passes through `src/proxy.ts`: valid token required, then path-prefix → role check (`/admin` → `ADMIN`, etc.), redirect to `/unauthorized` on mismatch.
6. The target route handler independently re-runs `getSession` + role check (defense in depth).

8.3 Tutor Matching

1. Learner submits `MatchCriteria` (subject slug, topics, grade, preferred time slots, class format) to `POST /api/learner/match`.
2. Handler checks `matchingEnabled`, loads candidate published `SCHEDULED` classes in the subject with sessions, enrollment counts, and the teaching tutor's certified topics.
3. `rankMatches()` scores each: +10 per matched topic, +5 per certified matched topic, +3 per session landing in a preferred slot, +6/+3/+2 for exact/adjacent/any grade, up to +2 soonness tiebreak; full-classes excluded.
4. Ranked results with human-readable `MatchReasons` return to the learner. If nothing scores well, the UI routes the learner to post a **class request** (`TopicRequest`), optionally directed at one tutor.
5. A tutor accepts an open request → `POST /api/tutor/topic-requests/[id]/accept` transactionally creates a `TutorClass` and moves the request `OPEN` → `ACCEPTED`; enroll/unenroll and class status changes drive it through `ENROLLED` → `FULFILLED` (or re-open). Each transition emits notifications.

8.4 Tutor Certification Assessment

1. Tutor requests certification for a (subject, topic) → `TopicCertification` row (`PENDING`).
2. Tutor starts a quiz: `assessmentPicker.pickQuestionIds()` selects N bank questions (`origin: BANK`) per the global `getAssessmentConfig()` (question count, pass %, min bank size).
3. `AssessmentAttempt` + `AssessmentAttemptItem` rows track the run; on submit, `gradeAttempt` computes `scorePercent` and sets `PASSED` / `FAILED`.
4. If `autoCertifyOnAssessmentPass` is on, a pass flips the certification to `CERTIFIED`; otherwise an admin reviews at `/admin/certifications` (`CERTIFIED` / `REJECTED` + note). Result notification is emitted.
5. Tutors may raise a `QuestionRequest` asking an admin to add bank questions for a thin topic.

8.5 Session Pre-/Post-Test (learner progress)

1. Tutor builds one `SessionTest` per `ClassSession` (unique) from bank + self-authored questions; publishing sets `PUBLISHED`.
2. Learner takes the *same* question set twice: a `PRE` attempt opens once the test is published; the `POST` attempt opens automatically when the tutor marks that session `COMPLETED` (`kind` lives on `SessionTestAttempt`, not the test).
3. `sessionTestAccess` authorizes each attempt against the session lifecycle; `sessionTestResults` computes per-question and per-learner `PRE` → `POST` deltas.
4. Results feed the tutor's charts, the learner's `/learner/progress`, and the admin read-only views. Diagnostic only — no pass/fail. `sessionTestsEnabled` is the kill switch.

9. Data Architecture

The schema has **28 models** and 17 enums. The database is MariaDB/MySQL; Prisma owns the schema and migration history.

9.1 Entity Groups

Group	Models
Identity & access	User, TutorProfile, PasswordResetToken, IdCounter
Taxonomy	Subject, Topic
Classes & sessions	TutorClass, ClassTopic, ClassSession, ClassEnrollment, ClassAppeal
Matching / requests	TopicRequest, TopicRequestTopic, TopicRequestSlot
Assessment — tutor certification	AssessmentQuestion, AssessmentOption, AssessmentAttempt, AssessmentAttemptItem, TopicCertification, QuestionRequest
Assessment — session pre/post tests	SessionTest, SessionTestQuestion, SessionTestAttempt, SessionTestAttemptItem
Platform / cross-cutting	PlatformSetting, Notification, AuditLog, ChatbotMiss

9.2 Normalization & Denormalization

- **3NF for the core.** One-to-many child tables (`ClassTopic` , `ClassSession` , `TopicRequestTopic` , `TopicRequestSlot` , option/item tables) rather than JSON blobs or repeating columns.
- **Deliberate denormalization of `topic`** . Topics are plain strings on ~7 child tables, not foreign keys to `Topic` . `Topic` is the editable catalogue and validation source (`topicExists()`); renaming a topic fans the new string out to all denormalized columns inside one transaction. Rationale: the app never had DB-level referential integrity on `subject / topic` , and a full FK rewrite carried a far larger blast radius for no operational gain (decision B in `subject-topic-management.md`).
- **Compound unique constraints** enforce business rules at the DB (e.g. one enrollment per (`classId` , `learnerId`), one `SessionTest` per `sessionId` , one certification per (`tutorProfileId` , `subject` , `topic`)).
- **Enum-backed status** columns (`AccountStatus` , `ClassStatus` , `TopicRequestStatus` , `AssessmentAttemptStatus` , `SessionTestStatus` , ...) keep lifecycle state valid without a lookup table.

9.3 Anonymous Identity Generation

`generateAnonymousId(role)` performs an atomic `prisma.idCounter.update({ data: { count: { increment: 1 } } })` — the increment is executed by the database, so concurrent registrations cannot collide — then zero-pads to `TUT-0001` / `STU-0001` . The same pattern (`generateClassCode`) issues `C-0001` class codes, upserting the counter row so it works on a fresh database.

9.4 Entity-Relationship Diagram

The authoritative ERD is generated from `prisma/schema.prisma` into `docs/erd.md` (Mermaid) on every `prisma generate` via `prisma-erd-generator` . Summary of the principal relationships:

```
User 1-0..1 TutorProfile
TutorProfile 1-* TopicCertification | AssessmentAttempt | TutorClass | QuestionRequest | ClassAppeal
TutorClass 1-* ClassTopic | ClassSession | ClassEnrollment ; TutorClass 1-* ClassAppeal
ClassEnrollment *-1 User (learner)
ClassSession 1-0..1 SessionTest
SessionTest 1-* SessionTestQuestion (→ AssessmentQuestion) | SessionTestAttempt
SessionTestAttempt 1-* SessionTestAttemptItem (→ AssessmentQuestion, AssessmentOption)
TopicRequest *-1 User (learner) ; 1-* TopicRequestTopic | TopicRequestSlot
TopicRequest *-0..1 TutorClass (fulfilledClass) ; *-0..1 TutorProfile (directedTutor)
AssessmentQuestion 1-* AssessmentOption ; *-1 User (createdBy) ; *-0..1 TutorProfile (owner)
AssessmentAttempt 1-* AssessmentAttemptItem
Subject 1-* Topic
AuditLog *-1 User (admin) ; Notification *-1 User ; ChatbotMiss *-0..1 User
```

10. Security & Privacy Architecture

10.1 Authentication

- NextAuth Credentials provider; passwords stored only as bcrypt hashes (cost 12).
- Stateless JWT sessions signed with `NEXTAUTH_SECRET`, 8-hour expiry, carried in an HTTP-only cookie.
- Password reset: single-use, hashed, expiring `PasswordResetToken`; link delivered by SMTP (`src/lib/mail.ts`), console fallback when SMTP is unset.

10.2 Authorization

- **Layer 1 — edge middleware** (`src/proxy.ts`): authenticated + path-prefix → role match for `/dashboard`, `/admin`, `/tutor`, `/learner`.
- **Layer 2 — route handler**: every API route re-derives the session with `getSession(authOptions)` and checks `session.user.role`, returning 401/403 on failure. UI affordances are never the control.
- **Resource-scoped checks**: handlers verify ownership (e.g. a tutor may only edit their own class, a learner may only open their own attempt) via `sessionTestAccess`, `topicRequestVisibility`, and explicit `where` clauses.

10.3 Double-Blind Anonymity (RA 10173)

Core non-negotiable constraint. Learners and tutors must never see each other's real name, e-mail, or contact info across the peer boundary.

- Every student receives a sequential anonymous ID at registration; peer-facing UI and API responses expose only `id` + `anonymousId` plus non-identifying attributes (grade level, section).
- Peer-facing Prisma queries use explicit `select` to exclude `firstName` / `lastName` / `email`.
- Admins can see real identities (required for moderation). The `showTutorRealNames` platform setting is an explicit, admin-gated, default-off override — not a normal code path.
- Real name / e-mail changes require an admin route; grade level + section are self-service because they are not identity anchors.

10.4 Account Lifecycle

`AccountStatus`: `ACTIVE`, `SUSPENDED` (optionally time-boxed via `statusExpiresAt`, auto-reactivated at next login), `BANNED`, `PENDING` (awaiting approval, cannot log in), `DECLINED` (registration rejected, reason surfaced on `/account-declined`). Classes have a parallel lifecycle (`ClassStatus`) with a tutor `ClassAppeal` path that notifies admins.

10.5 Auditability

Every admin moderation action (suspend/ban user, moderate class, review certification, resolve request, edit settings) writes an `AuditLog` row (`adminId`, `action`, `targetType`, `targetId`, `reason`, `timestamp`) through `src/lib/auditLog.ts`, surfaced read-only at `/admin/audit-log`.

11. Cross-Cutting Concerns

Concern	Mechanism
Input validation	Zod schemas in <code>src/lib/validations/</code> ; first error → HTTP 400 with <code>{ error }</code> .
Error handling	Typed status codes (400/401/403/404/409/500); server errors logged via <code>console.error</code> , generic message returned.
Feature flags	<code>PlatformSetting</code> key/value rows read through <code>src/lib/settings.ts</code> with typed keys and defaults; edited at <code>/admin/settings</code> .
Notifications	<code>Notification</code> rows created as side effects of lifecycle events; polled by the client bell in <code>PortalLayout</code> ; per-row read-on-click. No push/real-time transport.
Taxonomy resolution	<code>src/lib/subjects.ts</code> caches the <code>Subject / Topic</code> catalogue and falls back to the static <code>SUBJECT_TOPICS</code> map when the DB is unreachable (e.g. unit tests).
Caching	In-process caches for the class browser and subject catalogue; no external cache tier.
Mail	Nodemailer transport from env; console fallback keeps dev/offline usable.
Testing	Vitest; <code>@/lib/prisma</code> mocked via <code>vi.hoisted</code> , <code>getSessionSession</code> mocked; positive, 400, 401/403, 409, and 500 paths covered per route.
Theming	OKLCH DaisyUI token palette in <code>src/app/globals.css</code> ; components use semantic classes only.

12. Deployment & Infrastructure View

```
+-----+
| Browser | responsive web, no native app
+-----+
| HTTPS  |
+-----v-----+
| Node.js runtime | single process
| Next.js 16 server |
| - RSC + route handlers |
| - edge middleware |
| - NextAuth (JWT) |
+-----+-----+
| Prisma (MariaDB | SMTP (Nodemailer)
| adapter) | -> password-reset mail
+-----v-----+ +-----v-----+
| MariaDB | | SMTP server | optional; console fallback
| / MySQL | +-----+
+-----+
```

Runtime characteristics

- **One deployable unit** — `pnpm build` then `pnpm start`. No separate API service, worker, broker, or cache.
- **Stateless app tier** — sessions are JWTs, so the Node process holds no session state; horizontal scaling is possible but unnecessary at TRIS scale.
- **Single stateful dependency** — the MariaDB database. SMTP is optional.
- **Schema changes** — `prisma migrate deploy` against the versioned history in `prisma/migrations/`; seed via `pnpm exec tsx prisma/seed.ts`.

Required environment

Variable	Purpose
<code>DATABASE_URL</code>	MariaDB/MySQL connection string
<code>NEXTAUTH_URL</code>	Canonical app origin
<code>NEXTAUTH_SECRET</code>	JWT signing secret (long, random, never shared)
<code>SMTP_*</code> (host/port/user/pass/from)	Password-reset mail; omit to use console fallback

13. Source Tree Layout

```
katuwang/  
  prisma/  
    schema.prisma      28 models, 17 enums (source of truth)  
    migrations/        versioned SQL history  
    seed.ts            baseline data (id counters, admin, subjects...)  
  src/  
    app/  
      api/              ~55 REST route handlers  
        auth/[...nextauth]/ NextAuth handler  
        register/ | search/ | subjects/ | notifications/  
        admin/ | tutor/ | learner/ | classes/ | chatbot/ | dev/  
        admin/ tutor/ learner/   role portals (Server Component pages)  
        dashboard/           role dispatcher -> portal redirect  
        login/ logout/ register/ forgot-password/ reset-password/  
        pending-approval/ account-declined/ unauthorized/  
        globals.css          Tailwind 4 + DaisyUI 5 OKLCH theme  
        layout.tsx           root layout  
    components/  
      auth/ tutor/ learner/ admin/ charts/ chatbot/ providers/  
      PortalLayout, GlobalSearch, ...  
    lib/  
      auth.ts prisma.ts idGenerator.ts settings.ts  
      matching.ts browseRanking.ts  
      assessmentPicker.ts assessmentConfig.ts gradeAttempt.ts  
      sessionTestAccess.ts sessionTestResults.ts  
      moderation.ts auditLog.ts notifications.ts mail.ts  
      subjects.ts subjectTopics.ts  
      chatbot/ (classifier, intents, faq, recommend)  
      validations/ (Zod schemas)  
      __tests__/  
      proxy.ts             NextAuth edge middleware (RBAC guards)  
      types/               NextAuth session type augmentation  
    docs/                  reference, roles, plans, ERD, checklists  
    vitest.config.ts
```

14. Architecture Decision Record (Summary)

Full rationale lives in `docs/reference/decisions.md` and the per-feature plans under `docs/plans/`. The decisions that shaped this architecture:

#	Decision	Rationale
ADR-1	Modular monolith on Next.js App Router, not a separate SPA + API	One language, one deploy, minimal ops for a school with little technical capacity; RSC removes most client data-fetching code.
ADR-2	Three roles only; "Teacher Moderator" dropped	Owner decision (2026-09-03). Moderation folds into <code>ADMIN</code> ; avoids a fourth portal and guard set.
ADR-3	JWT session strategy (no DB session table)	Stateless app tier; school-day (8 h) lifetime; no session store to operate.
ADR-4	Two-layer RBAC (edge middleware + per-route re-check)	Defense in depth; middleware is coarse and fast, route handlers own resource-level authorization.
ADR-5	Anonymous IDs via atomic <code>IdCounter</code> increment	Race-free sequential IDs (<code>TUT-/STU-/C-</code>) without a UUID's opacity cost to users; satisfies RA 10173 double-blind.
ADR-6	Subjects/topics: catalogue tables + denormalized string columns (Path B)	Admin-editable taxonomy with a small blast radius; no FK migration; slug stays equal to the old enum value (lossless cast).
ADR-7	Global assessment config, not per-(subject, topic)	One <code>PlatformSetting</code> -backed set (question count, pass %, min bank size); the per-topic config table was removed.
ADR-8	Session pre/post-test scoped per <code>ClassSession</code> , one question set served twice (<code>kind</code> on the attempt)	Per-session granularity makes score change attributable to that session's material; identical question set makes per-question PRE → POST deltas meaningful.
ADR-9	Chatbot is deterministic intent matching, no LLM	Matches the thesis delimitation ("no free-form generative chat"); zero new runtime dependency; misses logged to <code>ChatbotMiss</code> to grow the FAQ.
ADR-10	No scheduler / queue / real-time transport	Keeps the deployment a single process; all work is request-driven; time-based reminders are explicitly out of scope.
ADR-11	No file upload / storage	Tutoring happens on school premises; removes a whole class of storage, quota, and moderation concerns.
ADR-12	Domain logic as pure modules in <code>src/lib/</code>	Unit-testable without HTTP/DB; keeps route handlers thin and readable.

15. Non-Functional Considerations & Known Limitations

Area	Current position	Extension point
Scalability	Single Node process + one MariaDB instance; ample for hundreds of concurrent students.	App tier is stateless (JWT) — can run multiple instances behind a load balancer; DB read replicas if ever needed.
Availability	No HA design; a single DB is the SPOF.	Managed MariaDB with automated backups / failover.
Notifications	Client-poll only; no time-based or push notifications.	Would require introducing a scheduler/queue (deliberately deferred, ADR-10).
Chatbot knowledge base	26 grounded FAQ entries; still pending real TRIS stakeholder interviews.	Grow from the <code>ChatbotMiss</code> table via <code>/admin/chatbot</code> .
Referential integrity of topics	Enforced in application code, not by FK.	Acceptable per ADR-6; a future FK rewrite (Path A) is possible but unplanned.
Observability	<code>console.error</code> logging + the audit log; no APM/metrics stack.	Structured logging / metrics exporter if operationalized beyond TRIS.
Rate limiting	None at the app layer.	Reverse-proxy rate limiting for auth and registration endpoints.
Internationalization	English UI with Filipino domain vocabulary; not localized.	Next.js i18n routing if bilingual UI is required.

16. Appendix A — Data Model Inventory

28 models in `prisma/schema.prisma`:

Model	Purpose
User	Every person; identity, credentials (hash), role, grade/section, consent, account status
PasswordResetToken	Single-use, hashed, expiring reset tokens
TutorProfile	1:1 extension of a tutor User; anchor for certifications, classes, attempts
TopicCertification	Per-(tutor, subject, topic) qualification state (PENDING/CERTIFIED/REJECTED)
IdCounter	Atomic counters for TUT-/STU-/C- sequential IDs
Subject / Topic	Admin-editable subject taxonomy + validation source
TutorClass	Course container: subject, target grade, capacity, location/link, status
ClassTopic	Topics covered by a class (child rows)
ClassSession	One scheduled meeting within a class, tackling one topic
ClassEnrollment	Learner ↔ class join with unique (classId, learnerId)
ClassAppeal	Tutor appeal against class moderation; admin-reviewed
TopicRequest / TopicRequestTopic / TopicRequestSlot	Learner “class request” ticket with topics and availability slots; public or directed
AssessmentQuestion / AssessmentOption	MCQ bank; <code>origin</code> BANK (admin) or TUTOR (self-authored)
AssessmentAttempt / AssessmentAttemptItem	A tutor's certification quiz run and its per-question answers
QuestionRequest	Tutor request for more bank questions on a topic
SessionTest / SessionTestQuestion	One diagnostic test per ClassSession; ordered question set drawn from the bank + tutor questions
SessionTestAttempt / SessionTestAttemptItem	A learner's PRE or POST run of a session test (<code>kind</code> on the attempt) and its answers
PlatformSetting	Key/value runtime feature flags & assessment config
Notification	Per-user in-app notification (type, message, link, readAt)
AuditLog	Immutable record of admin moderation actions
ChatbotMiss	Unmatched chatbot queries, for growing the FAQ

17. Appendix B — API Surface Summary

~55 route handlers under `src/app/api/`. Grouped by consumer:

Prefix	Consumer	Representative endpoints
/api/auth/*	All	[...nextauth] (login/session), forgot-password, reset-password
/api/register	Public	Learner + tutor registration (discriminated body)
/api/subjects, /api/search, /api/notifications, /api/chatbot	Authenticated (role-scoped results)	Taxonomy read, global quick search, notification list/read, chatbot turn
/api/learner/*	STUDENT_LEARNER	match, topic-requests, tutors, profile, progress, session-test-attempts/**
/api/tutor/*	STUDENT_TUTOR	classes/** (+ sessions/**, test/**, appeal), assessments/**, question-bank, questions/**, question-requests, topic-requests/[id]/accept, students, profile, topic-certifications
/api/classes/*	Learner (enroll)	classes (browse), classes/[id]/enroll
/api/admin/*	ADMIN	users/**, registrations/**, classes/**, class-appeals/**, topic-requests/**, certifications/**, assessment-questions/**, assessment-attempts/**, assessment-configs, question-requests/**, session-tests/**, subjects/**, topics/**, settings, reports, audit-log, chatbot-misses

All non-public routes: authenticate with `getServerSession(authOptions)` → check `session.user.role` → Zod-validate body → execute (transaction where multi-row) → return payload or `{ error }` with a typed status code.

End of document — Katuwang Finalized Architecture Design v1.0, 8 September 2026. Generated from the implemented system on `main`. The living sources are `prisma/schema.prisma`, `src/`, and the reference docs under `docs/`.